

Guía de Laboratorio: Análisis de la Cesta de Compras con Python

Bienvenidos a esta sesión práctica, donde aplicaremos técnicas avanzadas de **Retail Analytics** para transformar registros transaccionales brutos en conocimiento estratégico. El objetivo de esta guía es implementar el algoritmo Apriori, una herramienta fundamental para descubrir patrones de compra y asociaciones ocultas en grandes volúmenes de datos. Como futuros profesionales del área, es imperativo que no solo dominen la sintaxis de Python, sino que también desarrollen la capacidad analítica para interpretar métricas que guíen la toma de decisiones en el mundo real del retail.

1. Fundamentos Teóricos: El Algoritmo Apriori

Antes de comenzar la implementación, revisaremos los conceptos esenciales basados en la literatura técnica de minería de datos:

- **Market Basket Analysis (Análisis de la Canasta de Supermercado):** Técnica de extracción de información que identifica combinaciones de productos que suelen comprarse juntos (ejemplo: {pepinos, tomates} -> {aceite}). Su fin es optimizar el layout de tiendas y diseñar estrategias de venta cruzada.
- **Algoritmo Apriori:** Introducido en 1994 por Agrawal y Srikant, su nombre deriva del uso de **conocimiento previo** (*a priori*) de los conjuntos de elementos frecuentes. Es un algoritmo de aprendizaje no supervisado que genera reglas de asociación iterativamente.

Métricas Clave

- **Soporte (Support):** Indica la frecuencia relativa de un conjunto de elementos en el total de transacciones. Según la lógica del algoritmo, **añadir artículos a un grupo frecuente solo puede hacerlo menos frecuente**, nunca más (Propiedad de monotonía).
- **Confianza (Confidence):** Mide la probabilidad de que se adquiera el producto B dado que ya se compró el producto A.
- **Lift (Sustento):** Representa la fuerza de la asociación. Un Lift mayor a 1 indica que existe una **conexión real** entre los productos, superando lo que se esperaría por puro azar o por la popularidad individual de los artículos.
- **Propiedad Apriori:** Establece que, si un conjunto de elementos es frecuente, todos sus subconjuntos también deben serlo. Por el contrario, si un conjunto es infrecuente, todos sus superconjuntos también lo serán. Aunque el algoritmo Apriori es altamente adaptable y sencillo de entender, su proceso de iteración múltiple para generar candidatos puede ser **computacionalmente costoso e intensivo en memoria**.



UNIVERSIDAD DE LOS LAGOS

cuando se trabaja con datasets masivos. Es vital equilibrar los umbrales de soporte para optimizar el rendimiento.

2. Configuración del Entorno de Trabajo

Para este laboratorio utilizaremos Google Colab. Ejecute el siguiente bloque para instalar las dependencias necesarias.

```
#Instalamos las librerías necesarias para reglas de asociación y visualización
!pip install mlxtend squarify --quiet

#IMPORTANTE: Una vez terminada la instalación, vaya a:
# 'Entorno de ejecución' -> 'Reiniciar sesión' para aplicar los cambios.
```

3. Importación de Librerías especializadas

Utilizaremos mlxtend como motor principal para la generación de patrones frecuentes y reglas de asociación, y squarify para la representación gráfica jerárquica de los datos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import squarify
import warnings

#Herramientas de Machine Learning para Reglas de Asociación
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

warnings.filterwarnings('ignore')
%matplotlib inline

print(f"Librerías cargadas con éxito.")
print(f"Numpy version: {np.__version__}")
```

4. Carga y Preprocesamiento de Datos (Limpieza de NaNs)

Los datos de retail suelen venir en formato de lista de transacciones. Debido a que las canastas tienen tamaños variables (un cliente compra 2 productos y otro 20), la carga inicial en un DataFrame de Pandas genera valores nulos (NaN). Debemos limpiar estos datos para que el algoritmo procese listas de strings limpias.

```
#Carga del dataset (Asegúrese de subir Market_Basket_Optimisation.csv a Colab)
try:
```



UNIVERSIDAD DE LOS LAGOS

```
df_raw = pd.read_csv('Market_Basket_Optimisation.csv',
header=None)

transacciones = []
for i in range(df_raw.shape[0]):
    #Convertimos cada fila en una lista, eliminando valores 'nan'
    #El TransactionEncoder requiere una lista de listas de
strings.
    fila = [str(df_raw.values[i,j]) for j in
range(df_raw.shape[1]) if str(df_raw.values[i,j]) != 'nan']
    transacciones.append(fila)

print(f"Procesadas {len(transacciones)} transacciones con
éxito.")
except FileNotFoundError:
    print("Error: Por favor suba el archivo CSV al panel izquierdo de
Colab.")
```

5. Análisis Exploratorio: Visualización con Treemap

Antes de generar reglas, es fundamental entender qué productos dominan el volumen de ventas. Utilizaremos un Treemap para visualizar el "Top 20" de productos más frecuentes.

```
#Contamos frecuencias aplanando el dataset original
df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)
df_counts = df.stack().value_counts()
df_res = df_counts.to_frame(name='Frecuencia').reset_index()
df_res.columns = ['Producto', 'Frecuencia']

#Visualización del Top 20 Items
plt.figure(figsize=(12, 8))
top_20 = df_res.head(20)
squarify.plot(sizes=top_20['Frecuencia'], label=top_20['Producto'],
alpha=0.8, color=plt.cm.Spectral(np.linspace(0, 1, 20)))
plt.title('Market Basket Analysis - Top 20 Items Más Vendidos')
plt.axis('off')
plt.show()
```

6. Codificación Binaria (One-Hot Encoding)

El algoritmo Apriori requiere una matriz booleana donde las filas representan las transacciones y las columnas los productos. Se marca con un 1 (o True) si el producto existe en la compra y 0 (o False) si no. Utilizaremos el TransactionEncoder para esta transformación.

```
#Transformamos la lista de transacciones a una matriz binaria
te = TransactionEncoder()
```



UNIVERSIDAD DE LOS LAGOS

```
te_ary = te.fit(transacciones).transform(transacciones)
df_encoded = pd.DataFrame(te_ary, columns=te.columns_)

print(f"Dimensiones de la matriz codificada: {df_encoded.shape}")
#Ahora tenemos una columna por cada producto único en el dataset.
```

7. Generación de Reglas de Asociación

Procederemos a aplicar el algoritmo para encontrar itemsets frecuentes y luego derivar las reglas de asociación. Filtraremos los resultados para enfocarnos en relaciones que tengan un **Lift significativo**.

```
#Encontramos itemsets frecuentes con un soporte mínimo (ej. 1%)
frequent_itemsets = apriori(df_encoded, min_support=0.01,
                             use_colnames=True)

#Generamos reglas de asociación basadas en la métrica Lift
#Filtramos reglas con un Lift mayor a 1 para asegurar asociaciones no
aleatorias.
rules = association_rules(frequent_itemsets, metric="lift",
                          min_threshold=1)

#Ordenamos por Lift descendente para ver las reglas más fuertes
primero
rules = rules.sort_values('lift', ascending=False)

#Mostramos las 10 reglas con mayor fuerza de asociación
print("Top 10 Reglas de Asociación:")
print(rules[['antecedents', 'consequents', 'support', 'confidence',
            'lift']].head(10))
```

8. Consultas de Negocio: La Perspectiva del Gerente de Retail

Como analistas, nuestro valor radica en la interpretación de estos resultados. Responda las siguientes preguntas basándose en la lógica de los datos obtenidos:

1. **Análisis de Interdependencia:** Si observa un Lift elevado entre "herb & pepper" (hierbas y pimienta) y "ground beef" (carne molida), ¿qué decisión de *layout* o ubicación física debería tomar el gerente respecto al pasillo de carnes?
2. **Estrategia de Precios:** Si una regla {A} -> {B} presenta una **Confianza** del 70%, ¿cómo podría utilizar esta información para diseñar un "Pack" o combo promocional efectivo?



3. **Gestión de Inventario (Soporte):** ¿Por qué un producto con un Soporte muy bajo, a pesar de tener un Lift alto con otros productos, podría ser descartado por el algoritmo en primera instancia? ¿Qué riesgo corre el negocio si ignoramos estos productos "nicho"?

4. **Optimización de Ventas Cruzadas:** El Treemap muestra que el "mineral water" es el líder absoluto en frecuencia. ¿Es recomendable crear reglas de asociación basadas únicamente en este producto, o debería tratarse como un **"producto gancho"** (hook product) para atraer tráfico hacia asociaciones más rentables? Justifique su respuesta.